Obstacle Avoidance for Collaborative Robots Using Modulation Dynamical Systems Technique

Le Nam Chau[1], Pham Vo Khai Anh[1], Nguyen Thi Tuyet Mai[1], Pham Truong Phuong Nam[1], Minh-Tai Vo[2*]
[1]*Department of Automatic Control, HCMC University of Technology and Engineering (HCM-UTE)*, Vietnam
{22151054, 22151049, 22151112, 22151119}@student.hcmute.edu.vn
[2]*School of Science, Engineering & Technology, RMIT University*, Ho Chi Minh City 700000, Vietnam
*Corresponding author: S4140277@student.rmit.edu.au

***Abstract*—Collaborative robots (cobots) operating in shared human–robot workspaces require reactive, computationally lightweight obstacle avoidance to guarantee both safety and task continuity. This paper presents a simulation-based study of obstacle avoidance for the 6-DOF UR3 cobot (Universal Robots) using Modulation Dynamical System approach (MDS). A nominal autonomous dynamical systems (DS) with a stable target attractor is first designed to generate goal-directed end-effector motion. Reactive collision avoidance is then achieved by locally modulating the nominal velocity field through a modulation matrix constructed from each obstacle's geometric representation, deflecting the flow around obstacles while preserving asymptotic convergence to the attractor and avoiding the need for global path re-planning. The framework is evaluated in simulation across scenarios containing single and multiple static obstacles, where the cobot consistently reaches the target along collision-free trajectories. Limitations of the approach are analyzed and discussed, motivating directions for future work.**

***Index Terms*—Obstacle avoidance, Modulation Dynamical Systems, Collaborative robot, End-effector control, Reactive control.**

I. Introduction

In recent years, cobots have received much attention in the area of modern manufacturing ecosystems. Contrary to conventional industrial robots that perform tasks in isolated and structured workspaces, cobots work directly with human operators and are spatially close to one another. But the fact that such collaborative environments are dynamic introduces a new set of challenges. Cobot motion must react continuously, navigating unpredictable situations [1], [2] including human-like obstacles, known obstacles, and unknown environmental conditions. Hence, generating safe, reactive, and adaptive robot motion has become important concern in smart manufacturing environments. Especially, those designed to conform to relevant ISO safety standards [3].

Motion planning for obstacle avoidance in cobots is one of the main research areas in robotics and automated manufacturing. The capability to navigate safely and efficiently in environments containing static or dynamic obstacles is essential for the reliable operation of autonomous robotic systems. According to the technical report in [4], motion planning methods for obstacle avoidance can be categorized into five major clusters: reactive control methods, collision avoidance strategies, optimal control methods, data-driven approaches, and graph-based path planning techniques. The report also highlights the advantages of reactive control methods compared with other approaches in terms of global–local planning characteristics and computational efficiency (slow–fast computation). Among the reactive control approaches, two representative methods that have been extensively studied and developed in recent years are artificial potential fields (APFs) and DS/MDS. APF-based methods are widely used due to their intuitive formulation and computational efficiency. They generate robot motion by combining attractive forces that guide the robot toward a target with repulsive forces that push the robot away from obstacles [5], [6], [7]. However, APF-based methods suffer from several limitations, particularly the occurrence of local minima and oscillatory behaviors in cluttered environments. To address these issues, recent studies have explored learning-based approaches, such as reinforcement learning and deep neural networks, which improve adaptability to complex and uncertain environments [6], [8]. Another promising framework is based on DS and MDS, where robot motion is described using continuous-time differential equations instead of pre-computed trajectories. Such formulations enable smooth and reactive motion generation that adapts to disturbances and environmental changes. In this context, obstacle avoidance is achieved by modulating the vector field of a nominal dynamical system. In particular, the MDS approach modifies the nominal velocity field through a position-dependent modulation matrix, preventing collisions while preserving the attractor properties of the original system [9], [10], [11]. This property enables robots to respond quickly and efficiently to obstacles without requiring global path replanning.

Motivated by these considerations, this paper develops an MDS-based obstacle avoidance approach for collaborative robots and evaluates it on the UR3 cobot, modeled using the Unified Robot Description Format (URDF) [12], [13]. By coupling Cartesian-space motion generation with on-line modulation of a nominal dynamical system, the approach yields robust, real-time reactive behavior around obstacles while preserving asymptotic convergence to the attractor without resorting to global path replanning.

II. Dynamical System-based Collision Avoidance

A. Basic Notations

The mathematical symbols used in this paper are summarized in Table I as follows:

TABLE I: List of math notations.

Math Notations	Description
d	Dimension of the state space
N	Number of obstacles
x_c	Obstacle center position
$\varepsilon = x - x_c$	Relative position vector
$\|\varepsilon\|$	Euclidean distance to obstacle center
n	Normal vector
e	Tangent vector
Φ	Obstacle proximity function
K_p	Proportional gain
K_c	Convergence gain
R	Obstacle radius
a, b, c	Ellipsoid semi-axis lengths
$x_e,\ y_e,\ z_e$	Cartesian end-effector position

B. Robot Architecture

Table II summarizes the joint rotation axes, link offsets, motion limits, and maximum joint velocities of UR3 which adopted in this study. These parameters are extracted directly from the corresponding URDF model and manufacturer specifications [14] to preserve physical consistency.

TABLE II: Joint parameters of the UR3.

Joint	Rotation Axis	Offset (m)	Lower (rad)	Upper (rad)	Velocity (rad/s)
1	$Z(0,0,1)$	$z = 0.1519$	$-\pi$	π	2.16
2	$Y(0,1,0)$	$y = 0.1198$	$-\pi$	π	2.16
3	$Y(0,1,0)$	$y = -0.0925, z = 0.2437$	$-\pi$	π	3.15
4	$Y(0,1,0)$	$z = 0.2133$	$-\pi$	π	3.20
5	$Z(0,0,1)$	$y = 0.0851$	$-\pi$	π	3.20
6	$Y(0,1,0)$	$z = 0.0854$	$-\pi$	π	3.20

C. Obstacle Modeling

Each obstacle is modeled using a distance function based on a geometric representation approach. This method allows for a unified description of various 3D obstacle geometries, facilitating the modulation of the dynamical system. To simplify the computation, all distance functions are defined in a local reference frame centered at the obstacle. The distance function for an obstacle is defined as follows [9], [11]:

$$\Phi(\varepsilon) = \frac{\|\varepsilon\|}{R(\varepsilon)} \tag{1}$$

The Eq. 1 characterizes the geometric relationship between cobot and the obstacle as follows: $\Phi(\varepsilon) > 1$ indicates that the end-effector is located outside the obstacle, $\Phi(\varepsilon) = 1$ corresponds to the obstacle boundary, and $\Phi(\varepsilon) < 1$ represents a collision state.

The mathematical modeling of ellipsoidal obstacles is defined as:

$$\Phi_{\text{ellipsoid}} = \sqrt{\left(\frac{x_e}{a}\right)^2 + \left(\frac{y_e}{b}\right)^2 + \left(\frac{z_e}{c}\right)^2} \tag{2}$$

For vertical cylindrical obstacles, the distance function depends on the horizontal distance in the (x, y) plane due to invariance along the z-axis:

$$\Phi_{\text{cylinder}} = \frac{\sqrt{x_e^2 + y_e^2}}{R_{\text{cylinder}}} \tag{3}$$

The gradient of the distance function, which represents the normal direction to the obstacle surface, is given by:

$$\nabla\Phi = \left[\frac{\partial\Phi}{\partial x_e}; \frac{\partial\Phi}{\partial y_e}; \frac{\partial\Phi}{\partial z_e}\right] \tag{4}$$

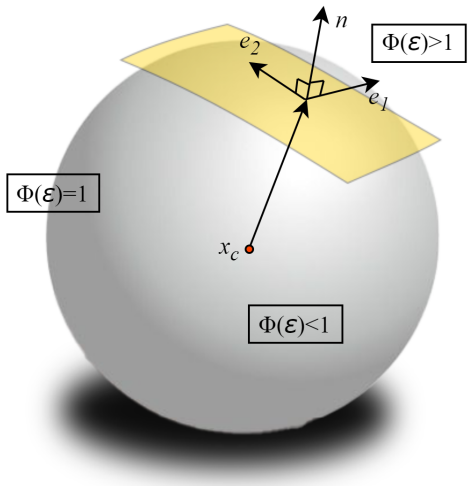


Fig. 1: Modeling of obstacle [9], [11].

D. Control Framework

1) Dynamical Systems: A DS in robot control is a continuous velocity vector field defined over the state space. The nominal velocity is described by:

$$\dot{x}_{ds} = f(x) \tag{5}$$

The DS is designed to be globally asymptotically stable, with a unique attractor located at the target position, ensuring convergence from any initial state without requiring prior trajectory planning.

$$f(x_t) = 0 \tag{6}$$

$$f(x) \neq 0, \forall x \neq x_t \tag{7}$$

2) Modulation Dynamical Systems: Main idea of MDS is to locally reshape the DS vector field when cobot approaches an obstacle, in a controlled manner that does not disrupt the overall structure of the system. This enhances the ability of the robot to adapt to unexpected obstacles without compromising its inherent stability. By applying a modulation matrix to the nominal system, the original dynamics are locally modified while preserving the global convergence properties [9], [11].

$$\dot{x_d} = T(\varepsilon) f(x) \tag{8}$$

Modulation matrix is presented as follows:

$$T(\varepsilon) = Q(\varepsilon)S(\varepsilon)Q(\varepsilon)^{-1} \tag{9}$$

Where

- Space decomposition matrix:

$$Q(\varepsilon) = \begin{bmatrix} n(\varepsilon) & e_1(\varepsilon) & ... & e_{d-1}(\varepsilon) \end{bmatrix} \tag{10}$$

- Eigenvalue matrix:

$$S(\varepsilon) = diag\begin{bmatrix} \lambda_1(\varepsilon) & \lambda_2(\varepsilon) & ... & \lambda_d(\varepsilon) \end{bmatrix} \tag{11}$$

with the coefficients $\lambda_1(\varepsilon) \leq 1$ compresses the normal direction, while $\lambda_m(\varepsilon) \geq 1,\ 2 \leq m \leq d$ expands the tangential direction.

$$\lambda_1(\varepsilon) = 1 - \frac{1}{\Phi(\varepsilon)},\ \lambda_m(\varepsilon) = 1 + \frac{1}{\Phi(\varepsilon)} \tag{12}$$

The behavior of the eigenvalues can be interpreted through two cases. When $\Phi(\varepsilon) \to 1^+$, the end-effector approaches the obstacle boundary. The normal eigenvalue $\lambda_1 \to 0$, suppressing motion toward the obstacle, while the tangential eigenvalues $\lambda_m \to 2^+$, enhance motion along the obstacle surface. When $\Phi(\varepsilon) \to \infty$, all eigenvalues converge to $\lambda_m \to 1$, and the modulation effect vanishes, causing the system to behave as nominal DS.

3) Extension to multiple obstacles: In the case of multiple obstacles, if all obstacles exert the same influence on the DS vector field, the resulting trajectory may become excessively distorted or lead to conflicting modulations in overlapping regions. Therefore, a weighting factor α is introduced to adjust the influence of each obstacle based on the distance between the Tool Center Point (TCP) and the $i-th$ obstacle [9], [11].

$$\alpha^i(\varepsilon^i) = \prod_{k=1,\, k\neq i}^{N} \frac{(\Phi^k(\varepsilon^k) - 1)}{(\Phi^k(\varepsilon^k) - 1) + (\Phi^i(\varepsilon^i) - 1)} \tag{13}$$

As the obstacle becomes closer, the weight increases, resulting in a stronger modulation effect. Accordingly, the components of the eigenvalue matrix are defined as follows:

$$\lambda_1(\varepsilon^i) = 1 - \frac{\alpha^i(\varepsilon^i)}{\Phi^i(\varepsilon^i)};\ \lambda_m(\varepsilon^i) = 1 + \frac{\alpha^i(\varepsilon^i)}{\Phi^i(\varepsilon^i)} \tag{14}$$

The final modulation is obtained as the product of the individual modulation matrices:

$$\dot{x}_d = \prod_{i=1}^{N} T_i(\varepsilon^i) f(x) \tag{15}$$

Alternatively, the modulation region can be adjusted by tuning the eigenvalues of the modulation matrix. A higher reactivity parameter (κ) leads to a larger deflection of the velocity field, allowing cobot to react earlier to the obstacle.

$$\lambda_1(\varepsilon) = 1 - \frac{1}{|\Phi(\varepsilon)|^{\frac{1}{\kappa}}};\ \lambda_m(\varepsilon) = 1 + \frac{1}{|\Phi(\varepsilon)|^{\frac{1}{\kappa}}} \tag{16}$$

In the obstacle avoidance model, safety radius (r_s) is used to create a safe spherical region around the end effector. Instead of treating the robot as an ideal point, this parameter enlarges the obstacle boundary by adding a safety margin to the distance function. As a result, the robot adjusts the motion earlier, before physical contact with the obstacle.

$$\Phi_r(\varepsilon) = \begin{cases} \Phi(\varepsilon) - r_s, & \text{if } \Phi(\varepsilon) > 1 + r_s \\ 1, & \text{if } \Phi(\varepsilon) \leq 1 + r_s \end{cases} \tag{17}$$

To further illustrate the control process, a diagram of MDS method is presented as follows:

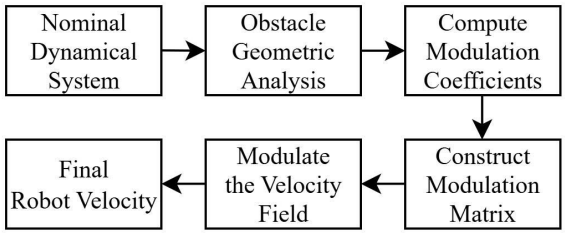


Fig. 2: Algorithm block diagram.

As shown in Fig. 2, the block diagram consists of 6-stage describing how robot motion is adjusted in environments with obstacles. First, system uses nominal DS velocity field to determine direction toward target point. When an obstacle is detected, cobot analyzes the obstacle geometry, computes distance, and determines the normal and tangential vectors (as illustrated in Fig. 1). Based on this distance, system defines an appropriate level of modulation (i.e. the closer the obstacle, the stronger the effect). Original velocity is then decomposed in local coordinate frame attached to obstacle, where the component along the normal direction is reduced to prevent collision, while the tangential component is amplified to generate an avoidance motion. After modulation, the velocity is transformed back into the Cartesian coordinate system and used as final control command, allowing cobot to reach target point safely and stably.

III. System Design

According to system workflow in Fig. 3, joint angles (q) are used in forward kinematics, implemented using built-in functions from the MATLAB Robotics Toolbox, to determine the TCP position in Cartesian space. Based on the current TCP position (x) and target position (x_t), the nominal velocity is calculated via DS function:

$$\dot{x}_{ds} = f(x) = -K_c(x - x_t) \tag{18}$$

Then, the desired velocity ($\dot{x}_d$) is calculated via MDS function. This process yields a 3×1 velocity vector in Cartesian space, redirected to achieve obstacle avoidance. Next, the desired joint velocity ($\dot{q}_d$) is derived from the TCP desired velocity using the pseudo-inverse of the Jacobian matrix:

$$\dot{q}_d = J^{\dagger}\dot{x}_d \tag{19}$$

This desired joint velocity is then fed into a proportional controller based on the error between it and the current joint velocity ($\dot{q}$) to calculate the joint acceleration according to the control law:

$$\ddot{q}_d = K_p(\dot{q}_d - \dot{q}) \tag{20}$$

External disturbances (τ_{dist}) are generated via keyboard input and modeled as a force acting on the TCP. These forces generate disturbance torques applied to the robotic system, resulting in joint accelerations that include the disturbance-induced torques. Subsequently, the ODE15s solver is used for time integration to compute the desired joint angles (q_d) and velocities. Finally, update UR3 robot corresponding to the new state within the simulation environment.

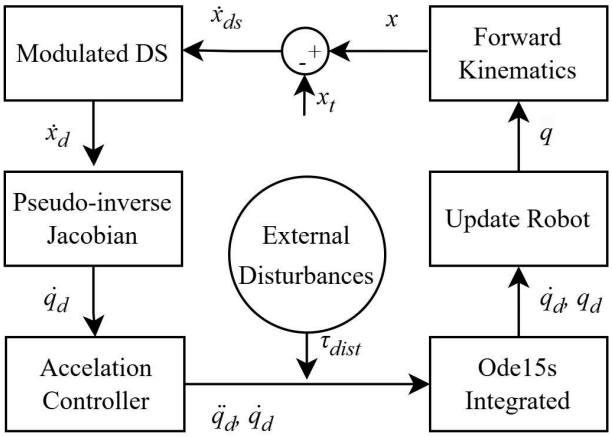


Fig. 3: Control scheme of the robotic system.

IV. Results

A. Nominal DS

The initial position is set to $[0.25; 0.3; 0.45]$ and the target position is set to $[0.3; -0.25; 0.4]$. The red line represents the DS trajectory, while the dashed black line denotes the TCP trajectory. According to Fig. 4, the TCP moves directly toward the attractor following the nominal DS trajectory, demonstrating the global convergence property of the DS in an obstacle-free environment.

B. Obstacle Avoidance Performance

1) Single Obstacle Scenario: In this scenario, a non-uniform ellipsoid obstacle is introduced with semi-axis lengths of $a = 0.05$ m, $b = 0.12$ m, $c = 0.07$ m. As illustrated in Fig. 5, the DS vectors near the obstacle are modified by the modulation matrix, which bends the vector field around the obstacle surface. After leaving the obstacle influence region, the DS returns to the original converging direction toward the attractor. This demonstrates that the MDS mechanism effectively avoids obstacles while ensuring convergence to the target. The final position reaches $[0.2984; -0.2406; 0.4030]$,

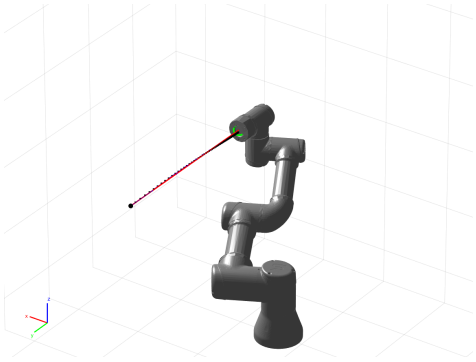


Fig. 4: TCP and DS trajectories without obstacles.

which is within the tolerance of 0.01 m. The convergence time is 4.65 s, while the total simulation time is 20.0695 s.

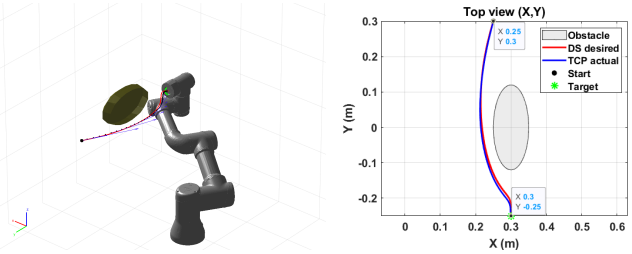


Fig. 5: TCP and DS trajectories with one obstacle.

2) Two Obstacles Without Weighting: When two obstacles are placed close to each other without applying obstacle weighting, the modulated DS may lead to a local minimum situation, as given in Fig. 6. Due to the strong overlap of the obstacle influence regions, the modulation matrices act simultaneously and generate conflicting velocity adjustments in the DS. In the overlapping region, the tangential directions induced by each obstacle become inconsistent, which reduces the resulting velocity and may create a spurious equilibrium or local minimum. This limitation has been reported in MDS when multiple convex obstacles are placed close to each other.

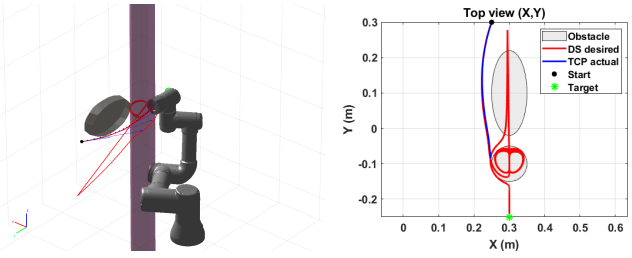


Fig. 6: TCP and DS trajectories with two unweighted obstacles.

3) Two Obstacles With Obstacle Weighting: To address the trapping issue, an obstacle weighting mechanism is introduced. The weights balance the influence of each obstacle according to its relative distance to the robot. In Fig. 7, the TCP is able to escape the local trap and continue moving toward the

attractor. The convergence time is $4.40\ s$, and the final position reaches $[0.2981, -0.2402, 0.3999]$, which is within $0.01\ m$ of the attractor. The total simulation time is $21.5479\ s$.

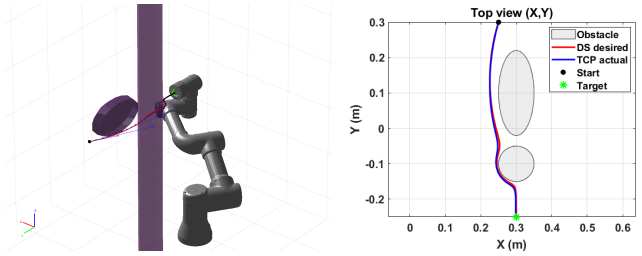


Fig. 7: TCP and DS trajectories with two weighted obstacles.

C. Extended Evaluation of the Implemented Method

1) External Disturbance: As presented in Fig. 8, the TCP trajectory temporarily deviates from the desired DS vector field under an external disturbance. However, due to the stability of the DS and the modulation mechanism, the TCP quickly recovers and continues converging toward the attractor. This result confirms the stability and robustness of the method.

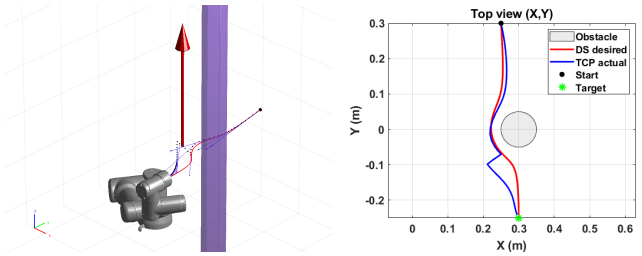


Fig. 8: TCP and DS trajectories under disturbance.

2) Moving Target Tracking: As illustrated in Fig. 9, the attractor is initially set to $[0.3; -0.17; 0.4]$ and extends to a time-varying point $x_t(t)$ moving along the z-axis. The robot continuously adapts its velocity direction to track the moving target while maintaining obstacle avoidance, demonstrating the fast reactivity and real-time adaptability of the method.

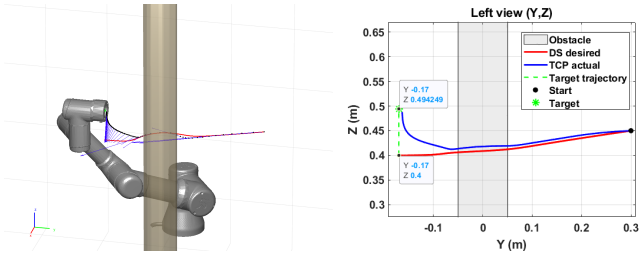


Fig. 9: TCP and DS trajectories with a moving attractor.

3) Moving Obstacle: To validate the real-time reactivity, Fig. 10 shows the obstacle avoidance process from $t = 0$ s to $t = 2.85$ s. As the obstacle moves along the y-axis, instead of maintaining the initial trajectory, the system adjusts its movement direction early to avoid the obstacle, successfully adapting to dynamic changes while continuing toward the attractor.

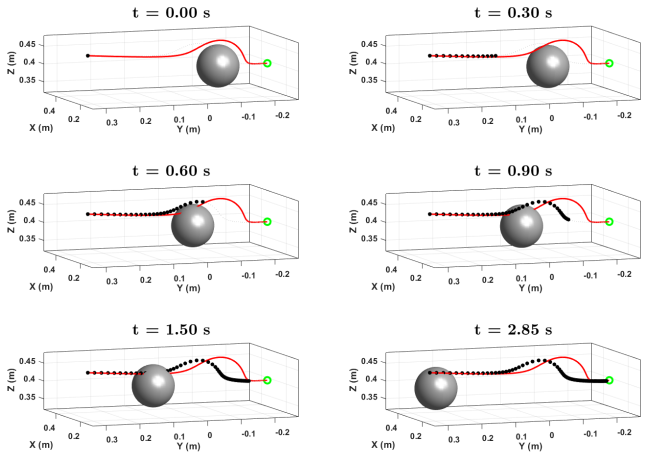


Fig. 10: TCP and DS trajectories with a moving obstacle.

4) Effect of the End-Effector Safety Radius: When r_S increases from $0.01\ m$ to $0.05\ m$, the DS trajectory is pushed further from the obstacles, as clearly observed in Fig. 11, with a minor deviation of approximately $0.001\ m$. The trajectory remains continuous and oriented toward the attractor. Table III shows the convergence time remains constant at $3.48\ s$, while the simulation time increases from $15.334\ s$ to $16.9181\ s$ due to the longer generated path. The converged position remains close to the attractor within allowable tolerance.

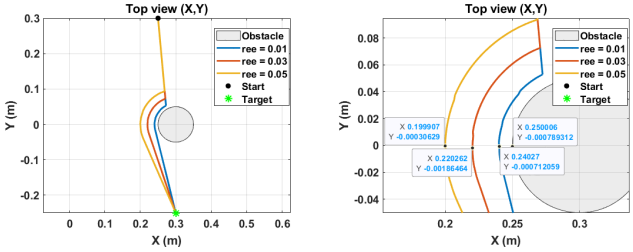


Fig. 11: DS trajectories under different Safety radius.

TABLE III: Results when varying the safety radius parameter.

r_s	0.01	0.03	0.05
Convergence time (s)	3.48	3.48	3.48
Simulation time (s)	15.334	16.0103	16.9181
Final position (m)	$\begin{bmatrix} 0.2954 \\ -0.2414 \\ 0.3985 \end{bmatrix}$	$\begin{bmatrix} 0.2948 \\ -0.2418 \\ 0.3982 \end{bmatrix}$	$\begin{bmatrix} 0.2945 \\ -0.2421 \\ 0.3981 \end{bmatrix}$

5) Effect of the Reactivity Parameter: According to Fig. 12 and Table IV, for $\kappa = 0.1$, modulation is activated only when the robot is extremely close to the obstacle (approximately $0.001\ m$). For medium κ (e.g., 1–3), avoidance initiates earlier, yielding smooth trajectories with respective clearances of $0.027\ m$ and $0.084\ m$. When κ is increased to 5, the robot detours widely around the obstacle (approximately $0.12\ m$),

enhancing safety but extending the path. As κ increases, the convergence time rises from $3.46\ s$ to $8.24\ s$, and the simulation time increases from $14.7749\ s$ to $39.7597\ s$ due to longer paths and computational costs. The final TCP position remains within allowable tolerance in all cases.

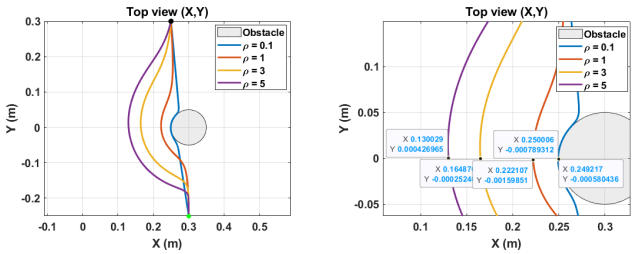


Fig. 12: DS trajectories under different Reactivity.

TABLE IV: Results when varying the reactivity parameter.

κ	0.1	1	3	5
Convergence time (s)	3.46	4.31	6.65	8.24
Simulation time (s)	14.7749	21.4811	33.1955	39.7597
Final position (m)	$\begin{bmatrix} 0.2957 \\ -0.2411 \\ 0.3986 \end{bmatrix}$	$\begin{bmatrix} 0.2976 \\ -0.2405 \\ 0.3986 \end{bmatrix}$	$\begin{bmatrix} 0.2987 \\ -0.2401 \\ 0.3993 \end{bmatrix}$	$\begin{bmatrix} 0.2994 \\ -0.2400 \\ 0.3997 \end{bmatrix}$

V. Discussion

Despite its advantages, MDS method has several limitations. First, it assumes convex obstacles; concave shapes or special geometric configurations can create spurious equilibrium points that locally trap the robot, requiring convex approximations or algorithmic extensions. Furthermore, the MDS considers collision avoidance only at the TCP. Collisions between the robot links and obstacles are not taken into account, and self-collision is not explicitly handled. This limitation becomes particularly important when deploying the approach in real-world experiments. Finally, the velocity is computed in Cartesian space and mapped to joint space using the Jacobian matrix. However, this mapping does not consider the full robot dynamics, which may affect the motion behavior when implemented on real robotic systems.

VI. Conclusion

This paper presents an MDS approach for obstacle avoidance in a UR3 cobot. The nominal linear attractor dynamics driving the system toward the target are modulated by a geometry-based matrix derived from obstacle information, resulting in nonlinear obstacle avoidance behavior. Simulation results under various scenarios, including multiple obstacles, parameter variations, the moving target and obstacle, and external disturbances, show stable convergence and smooth collision-free trajectories. For the works in the future, we target to extend this study by incorporating full-body collision checking to improve safety and implementing the algorithm on a real UR3 via RTDE protocol. In addition, a comparative study with methods in the class of deterministic reactive control such as APF [15], dynamic movement primitives [16], rotational obstacle avoidance method [17] will be conducted.

Acknowledgment

This research was funded by Ho Chi Minh City University of Technology and Engineering (HCM-UTE), Vietnam, under grant No. SV2026-273.

References

[1] Y. Wang, "The rise of collaborative robots: Technical and commercial insights," IDTechEx Research Article, Jul. 2025, accessed: 2026-02-24. [Online]. Available: https://www.idtechex.com/en/research-article/the-rise-of-collaborative-robots-technical-and-commercial-insights/33439

[2] M. Javaid, A. Haleem, R. P. Singh, S. Rab, and R. Suman, "Significant applications of cobots in the field of manufacturing," *Cognitive Robotics*, vol. 2, pp. 222–233, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667241322000209

[3] ISO, *ISO/TS 15066—Robots and robotic devices—Collaborative robots*, International Organization for Standardization Std., 2016, accessed: Dec. 23, 2025. [Online]. Available: https://cdn.standards.iteh.ai/samples/62996/37daa360128440ccbc94e9782c9e1748/ISO-TS-15066-2016.pdf

[4] L. Huber, "Exact obstacle avoidance for robots in complex and dynamic environments using local modulation," EPFL—Swiss Federal Institute of Technology Lausanne, Lausanne, Switzerland, Tech. Rep., 2024, accessed: Sep. 23, 2025. [Online]. Available: https://memento.epfl.ch/event/exact-obstacle-avoidance-for-robots-in-complex-and/

[5] C. Jiang, Y. Gao, and P. Yang, "Mobile robot obstacle avoidance based on improved artificial potential field method," in *2021 3rd International Conference on Robotics and Computer Vision (ICRCV)*, 2021, pp. 18–23.

[6] L.-S. Schneider, J. Peng, and A. Maier, "Robot movement planning for obstacle avoidance using reinforcement learning," *Scientific Reports*, vol. 15, no. 1, p. 32506, 2025.

[7] D. A. Dsouza, S. Shenoy, M. Wang, and A. R. Chowdhury, "A comprehensive safety architecture for human–robot collaboration in confined workspaces using improved artificial potential field," *Robotica*, vol. 43, no. 4, pp. 1373–1393, Apr. 2025.

[8] S. Luo and L. Schomaker, "Reinforcement learning in robotic motion planning by combined experience-based planning and self-imitation learning," *Robotics and Autonomous Systems*, vol. 170, p. 104545, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921889023001847

[9] S. M. Khansari-Zadeh and A. Billard, "A dynamical system approach to realtime obstacle avoidance," *Autonomous Robots*, vol. 32, no. 4, pp. 433–454, 2012. [Online]. Available: https://doi.org/10.1007/s10514-012-9287-y

[10] ——, "Realtime avoidance of fast moving objects: A dynamical system-based approach," in *Electronic proc. of the Workshop on Robot Motion Planning: Online, Reactive, and in Real-Time [IROS'2012]*, 2012.

[11] A. Billard, S. Mirrazavi, and N. Figueroa, *Learning for Adaptive and Reactive Robot Control: A Dynamical Systems Approach*. Cambridge, MA, USA: MIT Press, 2022.

[12] D. Tola and P. Corke, "Understanding URDF: A survey based on user experience," in *2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, 2023, pp. 1–7.

[13] ——, "Understanding URDF: A dataset and analysis," *IEEE Robotics and Automation Letters*, vol. 9, no. 5, pp. 4479–4486, 2024.

[14] A. Universal Robots and S. Energivej, "UR3 technical specifications," 2016.

[15] M. Elahres, M. Abbes, A. Fonte, and G. Poisson, "Artificial potential field apf-based obstacle avoidance technique for robot arm teleoperation," in *Proceedings of the 2023 27th International Conference on Methods and Models in Automation and Robotics (MMAR)*. IEEE, August 2023, pp. 222–227.

[16] M. Saveriano, F. Abu-Dakka, A. Kramberger, and L. Peternel, "Dynamic movement primitives in robotics: A tutorial survey," *The International Journal of Robotics Research*, vol. 42, no. 13, pp. 1161–1183, September 2023.

[17] L. Huber, J.-J. Slotine, and A. Billard, "Avoidance of concave obstacles through rotation of nonlinear dynamics," *IEEE Transactions on Robotics*, vol. 40, pp. 1983–2002, January 2024. [Online]. Available: https://doi.org/10.1109/TRO.2023.3344034